

EL-TA3BAN Pharmacy Management System



Student Project Information

- **Student Name:** Mohamed Abdelsalam
- **University:** British University in Egypt (BUE)
- **Faculty:** Computer Science
- **Course:** Programming in Java

Project Overview

A comprehensive pharmacy management system built in Java, featuring a complete healthcare workflow between patients, doctors, and pharmacists with secure wallet payment processing, prescription management, and inventory tracking. The system is now configured to automatically run in extended mode with all features enabled by default.

Features

- Multi-user authentication
- Prescription management
- Inventory tracking
- Digital wallet system
- Medical records
- Order processing
- Doctor-patient chat system

Project Structure

```
src/
├── data/           # Data access and persistence
├── gui/            # User interface components
│   ├── admin/      # Admin-specific UI
│   ├── components/ # Reusable UI components
│   ├── dashboard/  # Dashboard views
│   └── doctor/     # Doctor-specific UI
```

```
| navigation/ # Navigation components
| pharmacist/ # Pharmacist-specific UI
| screens/    # Main application screens
| theme/      # UI theming and styling
| models/     # Data models and entities
| resources/  # Application resources
| icons/      # Application icons
| config/     # Configuration files
| i18n/       # Internationalization
| services/   # Business logic and services
| utils/      # Utility classes

test/
| unit/       # Unit tests
| integration/ # Integration tests
| resources/  # Test resources

docs/
| api/        # API documentation
| user-guide/ # User documentation
| technical/  # Technical documentation
```

Build System

The project uses Maven for build automation. Key directories:

- **target/** - Build output
- **lib/** - External dependencies
- **pom.xml** - Maven configuration

Development

Prerequisites

- Java JDK 11 or higher
- Maven 3.6 or higher

Building

```
mvn clean install
```

Running Tests

```
mvn test
```

Running the Application

```
mvn exec:java -Dexec.mainClass="PharmacyGUI"
```

Documentation

- API documentation is available in [docs/api/](#)
- User guide is available in [docs/user-guide/](#)
- Technical documentation is available in [docs/technical/](#)

Setup in NetBeans

1. Download Project

- Clone from GitHub: <https://github.com/mohamedabdelsalam578/Pharmacy-Management-System>
- Or download ZIP and extract to a local directory

2. Open in NetBeans

- Open NetBeans IDE
- Go to File > Open Project
- Select the extracted project folder
- Click "Open Project"

3. Project Configuration

- Build path is already configured in nbproject/
- Source files are in src/
- Data files are in data/

4. Running the Project

- Right-click project in Projects window
- Select "Run" or press F6
- The program will automatically start in extended mode with all features enabled
- No command line arguments are needed

Test Credentials

- Admin: username "admin", password "admin123"
- Patient: username "amr", password "alice123"
- Doctor: username "drmohamed", password "doctor123"
- Pharmacist: username "fatima", password "pharm123"

Running the Application

The application can be run directly from NetBeans:

1. Right-click the project in the Project Explorer

2. Select "Run" or press F6
3. The system will automatically start in extended mode with all features enabled

Alternatively, you can run the project from the command line:

```
# Compile Java files to classes directory
javac -d classes src/*.java src/models/*.java src/services/*.java
src/utils/*.java

# Run the application from classes directory (runs in extended mode by
default)
java -cp classes PharmacyManagementSystem

# If you need basic mode for some reason, use the --basic flag
java -cp classes PharmacyManagementSystem --basic
```

Using the Application

After launching the application, you'll be presented with a main menu:

1. Login as Admin 🧑

- Default username: admin
- Default password: admin123
- Admin can manage medicines, users, and view reports

2. Login as Patient 👤

- Login with existing patient account
- Default patient: username: amr, password: alice123
- Patients can place/cancel orders, view prescriptions and medical reports

3. Login as Doctor 🧑

- Login with existing doctor account
- Default doctor: username: mohamed, password: doc123
- Doctors can create prescriptions and medical reports for patients
- Doctors can manage patient consultations and exchange messages

4. Login as Pharmacist 🧑

- Login with existing pharmacist account
- Default pharmacist: username: fatima, password: pharm123
- Pharmacists can process prescriptions and manage medicine inventory

5. Create Patient Account

- Register a new patient account in the system

6. Generate Documentation 📄

- Generate comprehensive system documentation in Markdown format
- Creates detailed documentation with system overview, classes, services, and data flow
- Documentation is saved to the `/docs` directory for easy reference

7. Exit

- Exit the application

Wallet Payment System

The EL-TA3BAN Pharmacy System features a comprehensive wallet-based payment system:

- **Patient Wallet:** Each patient has a personal wallet with:
 - Balance tracking
 - Transaction history
 - Deposit functionality
 - Payment processing
- **Security Features:**
 - Transaction logs with timestamps
 - Credit card integration with secure number masking
 - Balance validation to prevent overdrafts
- **Payment Options:**
 - Direct wallet payment
 - Credit card payment for orders
 - Transaction receipts
- **Functionality:**
 - Patients can deposit funds to their wallet
 - Pay for medicine orders directly from wallet
 - View complete transaction history
 - Link credit cards for payment

Data Persistence

All data is stored in text files and serialized objects in the `/data` directory:

- `admins.txt`: Admin user data
- `patients.txt`: Patient user data
- `doctors.txt`: Doctor user data
- `pharmacists.txt`: Pharmacist user data
- `medicines.txt`: Medicine inventory data
- `orders.txt`: Order transaction data
- `prescriptions.txt`: Prescription data
- `medical_reports.txt`: Medical report data

- consultations.txt: Doctor-patient consultation data
- messages.txt: Communication messages data
- data/wallets/: Directory containing wallet serialized objects

Main Classes

- **PharmacyManagementSystem**: Entry point to the application that automatically runs in extended mode
- **User**: Abstract base class for all user types (Admin, Patient, Doctor, Pharmacist)
- **Medicine**: Represents medicine items in the pharmacy inventory
- **Prescription**: Represents medical prescriptions created by doctors
- **Order**: Represents patient medicine orders
- **MedicalReport**: Stores patient medical information and diagnosis
- **Consultation**: Manages doctor-patient consultations
- **Pharmacy**: Central class representing the pharmacy entity
- **PharmacyService**: Main service integrating all user operations with complete healthcare workflow
- **AuthenticationService**: Handles secure user authentication with functional programming approach, password hashing and login attempt limiting
- **Wallet**: Manages patient wallet functionality for secure in-app payments
- **Wallet.Card**: Inner class that handles credit card functionality with secure number masking
- **ConsoleUI**: Provides colorful and interactive console UI elements with emojis and text formatting
- **FileHandler**: Manages data persistence using generic methods for loading and saving any entity type
- **SystemTools**: Helper utility class with methods for validation, formatting, security, and data manipulation

Object-Oriented Programming Concepts

This project was designed as an educational tool to demonstrate core OOP principles:

1. Inheritance

- **Class Hierarchy**: User (parent) → Admin, Patient, Doctor, Pharmacist (children)
- **Benefits**: Code reuse, extending functionality while maintaining common features
- **Example**: All user types inherit authentication and basic user attributes

2. Encapsulation

- **Data Protection**: Private fields with public getters/setters across all classes
- **Benefits**: Controlled access to data, validation of inputs, data integrity
- **Example**: Medicine class protects price, quantity and expiration data

3. Polymorphism

- **Method Overriding**: Each user type implements displayInfo() differently
- **Benefits**: Dynamic behavior at runtime, flexible code structure
- **Example**: Calling displayInfo() on any User reference shows role-specific information

4. Abstraction

- **Abstract Classes:** User class is abstract with concrete implementations
- **Benefits:** Enforces structure while hiding implementation details
- **Example:** Centralizes authentication but leaves specific behaviors to subclasses

5. Association Relationships

- **Composition:** Strong ownership (Patient owns Orders)
- **Aggregation:** Weaker relationships (Orders contain references to Medicines)
- **Benefits:** Models real-world relationships accurately
- **Example:** Prescription aggregates multiple Medicine objects

6. Modern Java Features

- **Streams API:** Used for efficient collection processing
- **Functional Programming:** Lambda expressions and function interfaces for cleaner, more concise code
- **Method References:** For cleaner callback implementation
- **Generics:** Type-safe collections and functional interfaces
- **Benefits:** More readable, maintainable and DRY code
- **Examples:**
 - Prescription's calculateTotalCost() uses stream processing
 - FileHandler's generic loadFromFile() and saveToFile() methods use Function interface
 - AuthenticationService's authenticate() method uses functional programming for finding users

Code Simplifications

The codebase has been significantly simplified while maintaining all functionality. Key improvements include:

- **FileHandler Improvements:** Generic methods for loading and saving data with reduced code duplication
- **Authentication Simplifications:** Functional programming approach for user authentication
- **UI Standardization:** Consolidated UI methods for consistent user experience
- **Cleaner Error Handling:** Improved exception handling throughout the system
- **Improved Documentation:** Detailed explanations of simplification rationale in SIMPLIFICATIONS.md

The simplifications follow key software engineering principles like DRY (Don't Repeat Yourself), Single Responsibility, and the Open/Closed Principle to create a more maintainable and extensible system.

Documentation Generation

The system includes an automated documentation generation feature that creates comprehensive Markdown documentation about the entire system:

How It Works

- Access the documentation feature from the main menu (option 6)
- The system dynamically analyzes the current state of all components
- Documentation is generated as a Markdown file in the `/docs` directory
- Each documentation file includes:
 - System overview and architecture
 - Detailed class descriptions and relationships
 - Service layer explanations
 - Data flow diagrams using ASCII art
 - Current system statistics
 - Healthcare workflow explanation

Benefits

- Always up-to-date documentation that reflects the current system state
- Comprehensive overview for new developers joining the project
- Educational tool showing real-world software documentation practices
- Useful reference for understanding the system architecture
- Markdown format for easy reading and version control

Interactive Console UI

The EL-TA3BAN Pharmacy System features an enhanced console user interface with:

- **Color Formatting:** Colorful text output for better visual appeal
 - Success messages in green
 - Warnings in yellow
 - Errors in red
 - Titles and headers in bold blue
- **Visual Elements:**
 - Loading spinners for processing operations
 - Progress bars for multi-step operations
 - Boxed UI elements with Unicode borders
 - Table formatting for data display
- **Interactive Elements:**
 - Emojis for visual context (🔍 for search, 💊 for medicines, etc.)
 - Animated transitions between screens
 - Styled menus with clear navigation options

The UI elements are implemented through the ConsoleUI utility class which provides a standardized interface for all system components to use.

Educational Comments

Throughout the codebase, you'll find detailed educational comments explaining:

- Object-oriented design principles applied in each class
- The reasoning behind design decisions
- Real-world analogies to software concepts
- Egyptian localization aspects and cultural references

These comments make the code not just functional but a learning tool for Java and OOP concepts.

License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.